GUIA VISUAL 006

ANATOMIA DE UM COMANDO

`rm arquivo com espaco.txt` *não apaga um arquivo: apaga três. Quem separou foi o shell, antes do programa existir.*

006
GUIAS LINUX

A LINHA, PALAVRA POR PALAVRA

`nmap`	`-sV`	`--top-ports`	`100`	`-oN`	`saida.txt`	`192.168.56.0/24`
COMANDO o que será executado	OPÇÃO CURTA um traço, uma letra	OPÇÃO LONGA dois traços, palavra	VALOR pertence à opção anterior	OPÇÃO com valor	VALOR arquivo de saída	OPERANDO o alvo do trabalho

Para o shell, todas as palavras depois do comando são iguais. Quem decide que `-sV` é opção e o resto é alvo é o **programa**, ao ler seu vetor de argumentos — por isso não existe sintaxe universal. Curtas sem valor se agrupam (`ls -lah`); `--` marca o **fim das opções** e salva de nome de arquivo que começa com traço: `rm -- -arquivo-estranho` . E há os rebeldes: `tar xzf` , `ps aux` , `dd if=` … `of=` … — sem traço nenhum.

COMO O SHELL DECIDE O QUE EXECUTAR

1. **PALAVRA-CHAVE** — `if` , `for` , `while` — é sintaxe, não programa
2. **ALIAS** — Apelido seu ou da distro. **NÃO VALE EM SCRIPT**
3. **FUNÇÃO** — Definida na sessão ou no arquivo de init
4. **BUILTIN** — `cd` , `echo` , `export` — dentro do próprio shell
5. **PATH** — Arquivo em disco, na ordem dos diretórios

`type -a <comando>` **mostra qual das cinco venceu** — e explica por que o terminal e o script se comportam diferente. O **diretório atual não está no** `PATH` : por isso `./script.sh` . Não é inconveniência, é segurança — com `.` no `PATH` , um arquivo chamado `ls` num diretório gravável vira escalada de privilégio. **Nunca acrescente** `.` **ao** `PATH` **.**

ZERO É SUCESSO — E É O QUE DÁ SENTIDO AO ENCADEAMENTO

CÓDIGO DE SAÍDA	SIGNIFICA
0	Sucesso — o único valor bom
1	Erro genérico (`grep` sem achar nada)
2	Erro de uso / arquivo inexistente
126	Existe, mas não é executável
127	Comando não encontrado — erro de digitação ou não instalado
130	Interrompido por `Ctrl+C` (128 + sinal 2)

`a && b`
Roda **b** só se **a** deu certo. **O padrão para tudo que pressupõe o passo anterior.**

`a || b`
Roda **b** só se **a** falhou — o tratamento de erro.

`a ; b`
Roda **b** sempre, dê no que der.

`a &`
Roda em segundo plano e devolve o prompt na hora.

`\rm · command rm`
Ignora o alias e chama o programa real.

Trocar `&&` por `;` já destruiu sistemas. Em `cd /tmp/trabalho ; rm -rf ./*` , se o `cd` falhar por um erro de digitação, o `rm -rf` executa **no diretório em que você estiver** — que pode ser o seu `~` . Escreva sempre `cd /tmp/trabalho && rm -rf ./*` . Mesma família de erro: em `rm -rf "$DIR"/` com `$DIR` vazia, a linha vira `rm -rf /` . E `sudo echo x > /etc/arquivo` **falha**: quem redireciona é o seu shell, não o sudo — use `echo x | sudo tee -a /etc/arquivo` .

RESUMO RÁPIDO

- ✓ Ler → separar por espaço → expandir → resolver → `fork` → `execve`. O programa só existe no fim.
- ✓ Espaço em nome quebra tudo: `"$arquivo"` entre aspas, sempre.
- ✓ `hash -r` (Bash) ou `rehash` (Zsh) quando o shell não acha o programa recém-instalado.
- ✓ O shell é só mais um programa: `ps -p $$ -o comm=` mostra qual está rodando de verdade.

NO KALI

O shell padrão é o **Zsh** desde a 2020.4 — para tudo desta ficha ele é idêntico ao Bash; use `rehash` no lugar de `hash -r`. A distro define aliases coloridos para `ls`, `grep`, `ip`: veja com `alias` e lembre que **script nenhum os enxerga**. Ferramenta clonada em `/opt` não está no `PATH` — faça um link em `/usr/local/bin`. **Cuidado com comando copiado de artigo:** dá para esconder um comando extra num bloco formatado. Cole num editor antes de executar — o shell obedece à linha, não à sua intenção.